

PTO: A Workload-driven Predictive Table Optimizer for Lakehouse Systems

VENKATA VAMSIKRISHNA MEDURI, IBM Research-Almaden, USA

DAVID KREISMANN, IBM, Germany

RONALD BARBER, IBM Research-Almaden, USA

BERTHOLD REINWALD, IBM Research-Almaden, USA

Data lakehouse architectures manage both structured and semi-structured data, often using disaggregated storage with volumes that can reach petabyte scale of data stored in open table formats such as Apache Iceberg. Due to the size and storage structure, traditional indexes are cumbersome to maintain resulting in the need for effective table organization to enable efficient retrieval of relevant data for analytical queries. To maximize skipping of irrelevant data while scanning large tables, lakehouse systems rewrite data files according to pre-specified partitioning columns, target file sizes, row group sizes, and bin-packing or sort strategies. Optimizing these parameters can enhance the skipping of irrelevant data during table scans and improve query performance significantly. State-of-the-art lakehouse systems often require these parameters to be manually specified by the user which is impractical due to the combinatorial search space of parameter values thereby severely impeding the usability of the existing table optimization features in these systems.

Conducting an exhaustive search to find the best combination of these parameters is impractical because these parameters are interdependent on each other, and rewriting a table with a single instantiation of all the four parameters can already take several hours at terabyte-scale. This comes with the additional complexity that optimal parameter value settings are query workload-sensitive as the filter predicates associated with the scan operators in the workload determine the skipping benefits we can get on a data layout. To overcome these challenges, we built a workload-driven predictive table optimizer for data lakehouses (PTO). It analyzes the filter predicates in the input query workload and reduces the initial search space by utilizing heuristics based on scan selectivity and query frequency in combination with the sort cluster sizes to filter out poorly performing partitioning columns and multidimensional sort column candidates. Thereafter, our PTO solution utilizes table sampling and Gradient Boosting Trees to discover the best combination of table optimization parameters. While our solution is applicable to lakehouse systems and open table formats which adopt similar parameterized layouts, we implemented PTO on Presto lakehouse engine to optimize Apache Iceberg tables. Our experiments show that PTO reduces the average workload latency by 11% on TPC-H and 36% on TPC-DS benchmarks at SF 10K while speeding up scan-intensive, long latency queries by 3.4× and 11× respectively.

CCS Concepts: • **Information systems** → **Physical data models**; • **Computing methodologies** → **Machine learning**; **Classification and regression trees**.

Additional Key Words and Phrases: Lakehouse systems; Table optimization; Data layout discovery; Apache Iceberg; Presto; Velox; Prestissimo; Apache Spark; Z-ordering; Good-Turing heuristic; Iterative sampling

ACM Reference Format:

Venkata Vamsikrishna Meduri, David Kreismann, Ronald Barber, and Berthold Reinwald. 2026. PTO: A Workload-driven Predictive Table Optimizer for Lakehouse Systems. *Proc. ACM Manag. Data* 4, 1 (SIGMOD), Article 67 (February 2026), 26 pages. <https://doi.org/10.1145/3786681>

Authors' Contact Information: Venkata Vamsikrishna Meduri, IBM Research-Almaden, San Jose, USA, vamsi.meduri@ibm.com; David Kreismann, IBM, Munich, Germany, David.Kreismann@ibm.com; Ronald Barber, IBM Research-Almaden, San Jose, USA, Ron.Barber@gmail.com; Berthold Reinwald, IBM Research-Almaden, San Jose, USA, reinwald@us.ibm.com.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2836-6573/2026/2-ART67

<https://doi.org/10.1145/3786681>

1 Introduction

Data lakehouses [41] manage both structured and semi-structured data and one of the primary reasons they can scale up to petabytes and exabytes of data is the decoupling of storage from the computational engine which means that query processing can happen in a separate engine from where the data is actually stored thereby enabling federated and distributed querying over multiple data sources. In particular, lakehouse engines adopt the open table format which is a metadata layer that adds structure on the top of raw data files to organize them into tables [24, 35, 36]. Examples of such open table formats are Apache Iceberg [3, 4, 27], Hive [38], Hudi [1, 2] and Delta tables [6]. Open table formats typically leverage metastores such as the Hive Metastore (HMS) [38] to store the metadata and statistical summaries about tables and schemas at various levels of granularity i.e, file, snapshot and partition-level for faster data access [40]. Data catalogs such as Unity Catalog [11] and Project Nessie [23] allow access to metastores by organizing data into schemas and adding layers of lineage, governance and discovery. Lakehouses can directly connect to semi-structured storage repositories such as Amazon S3 object store [7] and avoid Extract-Transform-Load (ETL) overheads associated with warehouses thus enhancing scalability. However, one of the current bottlenecks plaguing lakehouses is the complexity associated with the usage of traditional indexes. Building and maintaining an index over large-scale open data storage repositories is cumbersome thus requiring an alternative solution to achieve skipping of irrelevant data while running scan-intensive queries. Although open data formats like Hudi [26] support indexes, those are essentially *metadata* indexes built on column statistics and the retrieval method is skipping-based rather than direct pointers to the data used by traditional indexes. Existing works [26, 43] enable skipping of irrelevant data using *min-max* statistics and metadata indexing to accelerate scan-intensive queries. However, organizing the data optimally to maximize skipping benefits is challenging.

In this paper, we explore how to organize the tables in one such open table format, i.e., Apache Iceberg [3], and enable fast retrieval of data which is relevant to the scan filters in a given query workload. Towards this goal, we develop a lakehouse predictive table optimizer (PTO) that takes the query workload and the Iceberg tables as input to automatically detect the optimal layout parameters for each table comprising the partitioning scheme, target file size for the tables stored in a columnar (Parquet [42]) format, row group size of the Parquet files, and a multidimensional sort strategy. Thus, PTO reorganizes the underlying tables to speed up table scans in the query workload akin to what indexes are intended to achieve.

State-of-the-art lakehouses such as Dremio [12, 13], Databricks Delta Lake [17] and Amazon S3 Tables [7] support the OPTIMIZE feature which achieves compaction, i.e., combining several smaller Parquet files created during intermittent ingestion into larger Parquet files whose average file size is below the threshold of target file size. However, the target file size (TFS) of each Parquet file and row group size (RGS) indicating #row groups within a file need to be manually specified. Advanced techniques like AutoComp [25] additionally determine the granularity of compaction tasks to be at a partition or a snapshot-level and recommend identifying TFS from the query workload. Predictive optimization in Databricks [18] decides when to periodically re-optimize the Delta tables. Similarly, Snowflake automatic clustering [14] can also detect when to periodically re-cluster micropartitions but does not identify the sort columns. Databricks *liquid clustering* [20, 30] alleviates this issue by choosing clustering keys and replaces Z-ordering with alternative space-filling techniques like Hilbert curves. Amazon Redshift also automatically determines the sort order of the rows by exploiting the multidimensional filter predicates appearing in the query workload [19], and it does not assume access to underlying data. Our system, PTO, assumes access to underlying data besides the query workload but can holistically configure all the table optimization parameters comprising partitioning column, TFS, RGS and sort strategies. The reason to pick these four parameters to

be automatically configured has to do with the way Iceberg tables are organized. Iceberg tables are organized under partitions similarly to Hive tables but have additional benefits of manifest files and metadata information that support quick pruning of unnecessary partitions, files and row groups if the layout parameters are properly configured. More about the interdependence among these parameters and why PTO co-discovers them will be discussed in Section 2.

A naive exhaustive search on the parameter space is infeasible because the Iceberg tables need to be rewritten and the query workload should be run for each combination of candidate parameter values to discover the best configuration with the least runtime as the optimal table layout. Enumerating the entire search space in this manner can take days or months. Therefore, we solve this problem using a divide-and-conquer approach. We first reduce the overall search space by finding the candidate top-k partitioning columns and sort schemes with the highest filtering capability from the input query workload along with discretized target file sizes and row groups sizes. Next, we train an ML model on the skipping benefits estimated from a tiny fraction of candidate layouts as training data and predict the optimal layout yielding the highest expected skipping benefits. Following are our contributions.

- (1) We build a workload-driven predictive table optimizer (PTO) which discovers optimized layout parameters and enables skipping of data irrelevant to the query scan fragments.
- (2) We automatically configure four crucial parameters to accelerate scans on Iceberg tables - <partitioning column, target file size (TFS), row group size (RGS), sort strategy>.
- (3) Our novel divide-and-conquer strategies detect the top-k partitioning columns and top-k sort schemes in a workload-driven manner. To this end, we rank the power set of sort columns by their filtering capability and sort-cluster sizes because large sort-clusters allow for more data skipping. We leverage sampled tables to efficiently compute the sort-cluster sizes.
- (4) We train gradient boosting trees (GBT) on sampled tables to efficiently score limited #training configurations and predict the optimal configuration with the largest skipping ability.
- (5) While our solution is applicable to lakehouse systems and open table formats which adopt similar parameterized layouts, we implemented PTO on the top of Presto [34, 37] lakehouse engine for Apache Iceberg tables.
- (6) Our experiments show that PTO reduces the average workload latency by 11% on TPC-H [10, 15] and 36% on TPC-DS [16, 31] benchmarks at SF 10K while speeding up scan-intensive, long latency queries by 3.4× and 11× respectively.
- (7) In our experiments, we show the latencies of PTO ran in an offline manner; however, to support evolving queries and data, PTO is expected to be run in the background periodically.

2 Preliminaries

2.1 Interdependence among layout parameters

In this section, we describe how the data layout parameters are interdependent which justifies their co-detection.

- (1) There is a clear correlation between partitioning column and TFS. If the partitioning column leads to *too many* partitions, then we need to *appropriately* keep the file size *large enough* to ensure that we do not get too many files during table compaction. It is hard to quantify what too many or large enough means since it is dependent on the query workload and the amount of latency incurred by the scan operators during query execution. This is the reason for not relying on manual heuristics for co-discovery of parameters and opting for a predictive model to do so.
- (2) We also have a dependency between the sort strategy and RGS. If Z-ordering the table results in several smaller Z-clusters, this can lead to poor skipping benefits when the row group is

relatively larger. For example, if we have 4 to 10 Z-clusters per row group, this can lead to reasonably good skipping benefits. The best case scenario is when Z-cluster size $\geq$ RGS.

- (3) Small row groups result in long Parquet footers which hold the row group level metadata within a file. This in turn results in suboptimal scan latencies as a majority of the scan time would be spent on reading a huge amount of metadata from the Parquet file footers compared to the actual content of the file. This establishes an inherent correlation between TFS and RGS.

2.2 Problem Statement

Given a query workload Q , a set of Iceberg tables to be optimized T , a lakehouse engine L , our predictive table optimization tool PTO determines the optimal *data layout* which is a quadruple of the best combination of table parameters found for each table $t \in T$ i.e., $\langle \text{partitioning column } P(t), \text{sort order } S(t), \text{target file size } TFS(t), \text{row group size } RGS(t) \rangle$ such that the average number of rows skipped collectively over T while executing Q on L is maximized.

2.3 Design Goals

Our design goals are motivated by the fact that an exhaustive search for optimal layouts leads to prohibitive costs at TPC-H/DS scale factor 10K (10TB). Without a predictive model, exhaustively evaluating our shortlisted space of 360 candidate layouts (generation of candidate data layouts is discussed in Section 3.2) on a single table takes $360 \times (2.74 \text{ hours per rewrite} + 0.27 \text{ hours for scoring}) / 24 = 45$ days for TPC-DS and $360 \times 3.26 / 24 = 48.9$ days for TPC-H on our cluster with 4 machines (specifications are in Section 4.1.3). On a larger experimental cluster, it would be faster but still incurs significant latency. If we score as few as ~ 12 candidates, that takes 36 hours for TPC-DS and 39 hours for TPC-H but selects suboptimal layouts from such a tiny search space. Following are our design goals.

- (1) In contrast to prior work that optimizes a single, most beneficial data layout parameter such as sorting or clustering, we adopt a comprehensive approach to co-discover multiple data layout parameters. We conjecture that co-discovering these parameters enhances skipping benefits.
- (2) While prior work such as Ding et al. [19] does not assume access to the underlying data and purely relies on the query workload, we leverage both the data and the query workload to score layouts with respect to their skipping capabilities on data samples.
- (3) To alleviate the prohibitive optimization latencies, we propose a two-step approach which first reduces the candidate search space of layouts and then trains a *lightweight* predictive model to learn skipping benefits from as few as 12 training layouts.
- (4) The system is designed to operate on modest computational resources. We leverage a small experimental cluster to discover optimal data layouts for TPC-H/DS tables at scale factor 10K and selected tables from TPC-DS 30K within reasonable optimization latencies.

3 System Architecture

3.1 Overview of PTO

As shown in Figure 1, we use Hive Metastore (HMS) [38] to store all the schema information and metadata including the statistics for the Iceberg tables. We use the Spark-Iceberg library [5] to perform DDL operations such as table creation and alteration, and Spark procedures such as *rewrite_data_files* to apply the best data layout via compaction of Parquet files. We use Prestissimo comprising the Java Presto [34, 37] coordinator along with native C++ Velox [33] workers as the lakehouse engine to run the DML query workload consisting of SELECT queries. We utilize the Presto command line interface (CLI) to capture the query logs automatically generated upon executing the workload. The queries are executed on the original set of tables, termed as full tables

selectivity computed as $\frac{rowsOut}{tableCardinality} \times 100.0\%$. Here, rowsIn refers to #rows going into the scan operator and it is different from tableCardinality because the skipped rows are excluded from rowsIn. If the table is partitioned, rowsIn includes the rows from relevant partitions. Furthermore, the effect of file skipping and row group skipping i.e., excluding row count from irrelevant files and row groups is also expected to be captured while computing rowsIn. An example scan fragment is listed in Table 1. Note that a scan fragment may include predicates pushed down by the optimizer from higher-level operators in the query plan.

Table 1. Sample Scan Fragment on the TPC-H lineitem table

Key	Value
fragmentID	0
scanOp	ScanFilterProject
filterPredicate	(CAST(shipdate AS timestamp)) >= (TIMESTAMP'1998-03-01 00:00:00.000')
filterColumns	['shipdate']
tableCardinality	5999989709
rowsIn	5999989709
rowsOut	538678298
selectivity	8.98%

We use the extracted scan fragments to create scan fragment queries for each table. An example query based on Table 1 for TPC-H lineitem is `SELECT SUM(l_quantity) from lineitem WHERE l_shipdate >= date_parse('03/01/1998','%m/%d/%Y');`. These queries are used to evaluate the candidate data layouts on sample tables. Additionally, for each table, we also record #queries in which a column appears as a filtering column in its scan fragments. We term this as *filter column frequency*. We also compute the average scan selectivity across all the scan fragments a filter column appears in. Note that we are interested in the scan fragment selectivity and not the individual filtering capability of the column. This is because there may be filter predicates on other columns and their conjunction (or disjunction) with the predicates on the current filter column of interest contributes to the overall scan selectivity. Regardless of the individual filtering capability of the column, its presence in multiple highly selective scan fragments across the entire query workload is deemed valuable.

3.2.2 Top-k Partitioning Columns. Based on the scan fragments extracted from the query workload, we shortlist the top-k categorical columns ('INT', 'VARCHAR' and 'DATE' types are allowed) with the highest filtering capability. To this end, for each candidate partitioning column, we determine the filtering capability based on its filter column frequency which is #queries in which the column appears in the scan filter predicate, and the average scan selectivity across all the scan fragments in queries where the column appears as a filtering column. We perform a <major, minor> sort of all the categorical columns in descending order upon <filter column frequency, (100 - average scan selectivity)%> and pick the top-k columns. We provide guardrails to avoid too few or too many partitions by providing configurable parameters for the expected number of minimum and maximum partitions, and the minimum cardinality of the table for it to be eligible for partitioning. Our default settings for all these parameters including top-k depend on the expected size of the candidate data layouts and are listed in Section 4.1.3. Besides the top-k partitioning columns shortlisted, we always include NULL as an extra candidate to evaluate the absence of a partitioning column in which case the table is not partitioned. PTO estimates #partitions for a partitioning column from the number of distinct values (NDV). We compute NDV from value_counts and null_value_counts in tablename\$files which is an Iceberg metadata table for the current snapshot. We leverage record_count and file_size_in_bytes to estimate table cardinality and file sizes.

Algorithm 1 `getCandSortSchemes(queries, table, colSet, topk, sampleRatemin)`

```

1: filterColFreq ← getFilterColFreq(colSet, queries)
2: avgScanSel ← getAvgScanSelectivities(colSet, queries)
3: shortListedCols ← sortColSet(colSet, filterColFreq, avgScanSel, DESC, threshold)
4: if |shortListedCols| == 0 then
5:   return “BIN-PACKING”
6: else
7:   colPowerSet ← getPowerSet(shortListedCols)
8:   avgDistSortClustSizes ← getAvgDistSortClustSizes(sampleRatemin, table, colPowerSet)
9: end if
10: scores ← scorePowerSet(colPowerSet, avgDistSortClusterSizes, filterColFreq, avgScanSel)
    {normalized equi-weight score}
11: sortColList ← arg_maxcols(scores)[:topk]
12: candSortSchemes ← {}
13: for candCols ∈ sortColList do
14:   if DECIMAL ∈ candCols.dataTypes then
15:     candSortSchemes.append(“SORT”, candCols) {major-minor multidimensional sort}
16:   else
17:     candSortSchemes.append(“SORT”, ZORDER(candCols)) {Z-ordering}
18:   end if
19: end for
20: return candSortSchemes

```

3.2.3 Top-k Sort Schemes. Algorithm 1 describes how candidate sort schemes are obtained for a table. For a given table with a set of columns *colSet*, we first shortlist columns with high filtering capability by sorting *colSet* on its filter column frequency and average scan selectivity similarly to the partitioning columns and imposing a pre-specified *threshold* (lines 1-3 in the algorithm). The default *threshold* value is listed in Section 4.1.3. If the *shortListedCols* is empty, we return bin-packing as the candidate sort scheme (line 4).¹ Otherwise, we compute all possible $2^{|shortListedCols|} - 1$ subsets of the set of shortlisted columns using the standard power-set definition (line 7). Next, we obtain the average distinct sort cluster size for each column subset in the power set (line 8) and score the subset using a weighted combination of the normalized values of average distinct sort cluster size, filter column frequency and the average scan selectivity by assigning equal weights to all the three scoring criteria (line 10). We omitted an algorithm for *scorePowerSet* in the interest of space constraints because it is fairly straightforward. Finally, we pick the top-k sort column sets with the highest score (line 11). Note that we compute average distinct sort cluster size because simply computing the average over sort cluster sizes will bring very low sort cluster sizes due to the presence of a long tail of small clusters. Skipping ability is maximized by the larger clusters and therefore, we use average distinct sort cluster sizes for scoring. We either apply major-minor ordering or use Z-ordering for multidimensional sort depending on whether a “DECIMAL” column exists among the candidate sort columns (lines 13-18) because Z-ordering on DECIMAL data type is not supported by Spark-Iceberg [5].

3.2.4 Computing the Average Distinct Sort Cluster Sizes. As listed in line 8 of Algorithm 1, the average over distinct sort cluster sizes is obtained for each subset comprising the candidate sort

¹Bin-packing is cheaper to execute than sorting but PTO does not include final compaction times while scoring the sort strategies. A possible extension to scoring mechanism could combine estimated compaction latencies with skipping benefits.

Algorithm 2 getAvgDistSortClustSizes(sampleRate_{min}, table, colPowerSet)

```

1: avgSortClustSizes  $\leftarrow \{\}$ 
2: for sortCols  $\in$  colPowerSet do
3:   sampleRatecur  $\leftarrow$  sampleRatemin
4:   while true do
5:     sample  $\leftarrow$  getBernoulliSample(sampleRatecur, table)
6:     distClustSizessamp  $\leftarrow$  getDistinctClustSizes(sample, sortCols)
7:     avgDistClustSizesamp  $\leftarrow \frac{\sum \text{distClustSizes}_{\text{samp}}}{|\text{distClustSizes}_{\text{samp}}|}$ 
8:     numSingletons  $\leftarrow$  getSingletons(distClustSizessamp)
9:     scaleUpRate  $\leftarrow \min(\frac{|\text{distClustSizes}_{\text{samp}}|}{(|\text{distClustSizes}_{\text{samp}}| - \text{numSingletons})}, \frac{|table|}{|sample|})$  {Good-Turing Heuristic}

10:    numDistClustorig  $\leftarrow \min(\text{scaleUpRate} \times \text{numSingletons} + |\text{distClustSizes}_{\text{samp}}| - \text{numSingletons}, |table|)$ 
11:     $\delta \leftarrow \frac{|table|}{\text{numDistClust}_{\text{orig}}^2}$ 
12:    avgDistClustSizeorig  $\leftarrow \text{avgDistClustSize}_{\text{samp}} \times (\delta \times \frac{|table|}{|sample|} + (1 - \delta) \times \frac{\text{numDistClust}_{\text{orig}}}{|\text{distClustSizes}_{\text{samp}}|})$ 
13:    if ( $\frac{\text{numSingletons}}{|\text{distClustSizes}_{\text{samp}}|} < 0.95$  OR  $\frac{|sample|}{|table|} > 0.95$ ) then
14:      avgSortClustSizes[sortCols]  $\leftarrow$  avgDistClustSizeorig
15:      break
16:    else
17:      sampleRatecur  $\leftarrow 10.0 \times \text{sampleRate}_{\text{cur}}$ 
18:    end if
19:  end while
20: end for
21: return avgSortClustSizes, sampleRatecur

```

columns in the power set. An example aggregate query to compute the average distinct sort cluster size on a store_sales sample for a candidate set of sort columns can be written as “SELECT AVG(DISTINCT sortClustSize) AS avgDistClustSize_{samp} FROM (SELECT COUNT(*) AS sortClustSize FROM store_sales TABLESAMPLE BERNOULLI(sampleRate) GROUP BY ss_sold_date_sk, ss_store_sk);”. Sampling is required because running this query can incur a non-trivial latency on the entire table. There are two challenges here: (1) setting the sample rate, and (2) scaling up average distinct sort cluster size obtained on the sample table to the original table. A naive usage of $\frac{\text{avgDistClustSize}_{\text{samp}}}{\text{sampleRate}}$ to estimate avgDistClustSize_{orig} leads to large errors. Figure 2 shows # distinct clusters on the sample ($|\text{distClustSizes}_{\text{samp}}|$) and the average distinct sort cluster size (avgDistClustSize_{samp}) at three different sampling rates of 0.001%, 0.01% and 0.1% in a toy example limited to 5 groups. We can see a long tail of singleton clusters at lower sampling rates (4 out of 5 clusters at 0.001%) which gets relatively shorter with larger sampling rates (1 out of 5 clusters at 0.1%). This is because most of the singleton clusters are not *truly* singletons. Those groups are just undersampled at lower sampling rates leading to smaller group sizes. We correct this by appropriately scaling up avgDistClustSize_{samp} using a Good-Turing heuristic [29, 32] to iteratively estimate avgDistClustSize_{orig} by increasing the sample rate until convergence.

Algorithm 2 breaks down the estimation of average distinct sort cluster size into two parts. First, the number of distinct clusters is estimated followed by the computation of average cluster size.

SELECT *ss_sold_date_sk*, *ss_store_sk*, COUNT(*) AS *sortClustSize* FROM *store_sales*
 TABLESAMPLE BERNOULLI (0.001) GROUP BY *ss_sold_date_sk*, *ss_store_sk* LIMIT 5;

<i>ss_sold_date_sk</i>	<i>ss_store_sk</i>	<i>sortClustSize</i>		<i>sortClustSize</i>	<i>sortClustSize</i>
2451098	700	1		10	100
2450972	386	1		1	3
2451685	280	2		25	250
2451997	550	1		1	15
2452497	664	1		1	1

Sampling rate	0.001%	0.01%	0.1%
$distClustSizes_{samp}$	2	3	5
$avgDistClustSize_{samp}$	1.5	12	73.8

Fig. 2. Sort Cluster Sizes for sampled TPC-DS *store_sales*

Note that we can also use APPROX DISTINCT in Presto [21] (instead of Good-Turing) to estimate the approximate count of distinct sort clusters. In any given sampling iteration, $distClustSizes_{samp}$ and $avgDistClustSize_{samp}$ (lines 6 and 7 in Algorithm 2) are computed on a Bernoulli sample of the table created using the current sample rate $sampleRate_{cur}$, which is initialized at a configurable parameter $sampleRate_{min}$. Then, the Good-Turing heuristic [29, 32] determines the bloating factor or the $scaleUpRate$ to estimate the actual distinct count of these clusters (which are *falsely* appearing as singletons due to lower sampling rate, as shown in figure 2) in the original table (line 9). This gives us the overall count of *singleton* clusters in the entire table as $scaleUpRate \times numSingletons$ while not bloating the count of the non-singleton clusters (line 10). The reasoning here is that non-singleton clusters are sufficiently sampled compared to singletons which is also reflected in their distinct cluster count and hence, we need not overestimate their count. Once we get the number of estimated distinct clusters of all sizes on the original table ($numDistClust_{orig}$), we divide it by $|distClustSizes_{samp}|$ to get the scale-up rate and multiply it with the estimated average distinct cluster size on the sample, $avgDistClustSize_{samp}$ (line 12) to get $avgDistClustSize_{orig}$. We in fact use a weighted combination with another term $avgDistClustSize_{samp} \times \frac{|table|}{|sample|}$ to account for errors in the estimation of $numDistClust_{orig}$ (line 12). Finally, we stop refining $avgDistClustSize_{orig}$ when at least one of the two convergence criteria is met: (a) the percentage of singletons is below a threshold (95% being an acceptable long tail), or (b) the sampling size is close to the full table cardinality ($> 95\%$ of $|table|$). For all the enumerated candidates of column combinations in the power set of columns, we observe case (a) in practice. Case (b) is included for the sake of algorithmic completeness. The reason for observing case (a) consistently is that 95% tail is long enough for sort cluster size estimation using Good Turing heuristic to converge earlier (i.e., below 0.1% upper bound of sample rate for all the SF 10K tables).

3.2.5 Discretizing TFS and RGS. We set the lower and upper bounds of target file size (TFS) as configurable parameters in PTO, with default values listed in Section 4.1.3. We discretize $[TFS_{min}, TFS_{max}]$ by choosing all powers of 2 such that $\forall x$, if $TFS_{min} \leq 2^x \leq TFS_{max}$, we add 2^x MB to the list of candidate TFS. For each candidate TFS, we obtain a set of candidate row group sizes (RGS) as $\frac{TFS}{2^i}$ where i is an integer and $i \in [1, 5]$ and additional candidate RGS=TFS with one row group per file. Thus, we allow between 1 and 10 row groups per Parquet file. As mentioned earlier in Section 2.1, too many row groups will lead to long metadata footers in a Parquet file which in turn lead to poor scan latencies. While we discretized file sizes, we also compared with a Bayesian hyperparameter optimization (HPO) baseline that uses continuous TFS values.

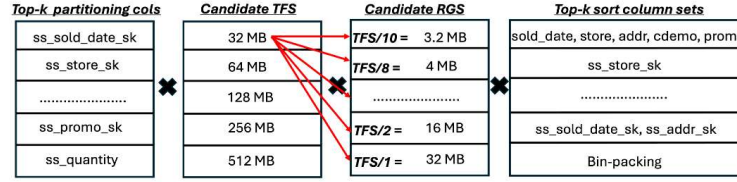


Fig. 3. Candidate Data Layouts for TPC-DS store_sales

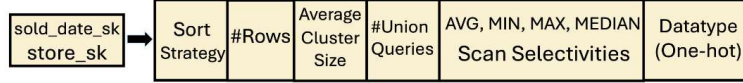


Fig. 4. Feature representation for a sample sort scheme

3.3 ML-based Detection of Best Data Layout

Figure 3 shows an illustration of the candidate space of layouts created for the TPC-DS store_sales table. Note that we included the option of not partitioning in the top-k candidate list of partitioning columns. We use 32MB and 512MB as the minimum and maximum TFS bounds while the candidate RGS are shown only for 32MB by dividing a Parquet file into 1,2,4,6,8,10 row groups. The sort strategies can either be major-minor sort or Z-ordering on a shortlisted column set or can use bin-packing. Even with a handful of shortlisted top-k candidates for each table optimization parameter and the search space containing only a few hundreds of data layouts, it can take a couple of months to perform an exhaustive search. To avoid this, we leverage ML models to find the best data layout among the shortlisted candidate layouts. This forms the second step of our two-step solution described in Section 3.1.

Algorithm 3 estimateSkippingBenefits(dataLayout, scanFragQueries, sampleTable, table)

```

1: samplingRate  $\leftarrow \frac{|sampleTable|}{|table|}$ 
2: downscaledTFS  $\leftarrow$  samplingRate  $\times$  dataLayout.TFS
3: downscaledRGS  $\leftarrow$  samplingRate  $\times$  dataLayout.RGS
4: if dataLayout.partitionCol  $\neq$  NULL then
5:   sampleTable  $\leftarrow$  CTAS(sampleTable, dataLayout.partitionCol) {partitioned copy of sampleTable}
6: end if
7: sampleTable  $\leftarrow$  rewrite(sampleTable, dataLayout.sortScheme, downscaledTFS, downscaledRGS)

8: skippedRowCount  $\leftarrow$  getNumRowsSkipped(sampleTable, scanFragQueries)
9: return skippedRowCount

```

3.3.1 Feature Vector Creation. We represent each data layout as a feature vector which is a concatenation of the numerical representation for each of the four components - partitioning column, TFS, RGS, sort columns. While TFS and RGS are used as they are in the feature vectors, deriving the features for partitioning column and sort columns is non-trivial. Figure 4 shows how the feature representation is derived for a sort scheme. We first represent the sort strategy as 0/1/2 depending on whether we use bin-packing, major-minor multidimensional sort or Z-ordering. This is followed by the dimensions for table cardinality, average distinct Z-cluster size, #queries which contain